

Project Technical Report: Web-Based Pan-Tilt Camera Control System with Raspberry Pi

Real-Time Video Streaming and Dual-Axis Servo Motor Control on Raspberry Pi

Project Team:

- Muhammed Emin Korkunç
- Nurettin Süleymanoğlu
- Faruk Kerem Bedir

Platform: Raspberry Pi (Linux / Python 3)

Web Framework: Flask
Microframework

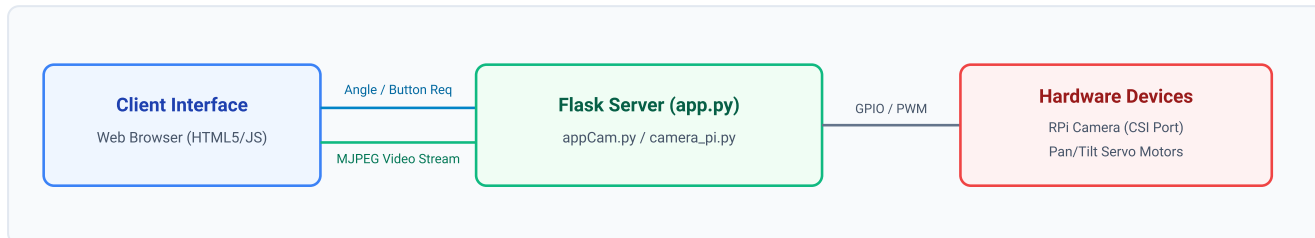
Protocol: HTTP / MJPEG Stream & REST

1. EXECUTIVE SUMMARY & SCOPE

This project aims to deliver low-latency, real-time video streaming on embedded architectures while providing remote dual-axis (pan and tilt) orientation control through a modern web browser. The implementation utilizes a Raspberry Pi single-board computer, an official Raspberry Pi Camera Module, and a two-axis gimbal driven by micro servo motors. The resulting solution demonstrates efficient edge-computing practices for remote surveillance, IoT robotics, and interactive monitoring.

2. SYSTEM ARCHITECTURE & DATA PIPELINE

The architecture comprises three core tiers: video ingestion, motion control, and client presentation. The Flask backend server delivers video frames continuously as a multipart MJPEG stream over HTTP, while processing incoming asynchronous motion requests to compute pulse-width modulation (PWM) signals routed directly to the GPIO pins.



3. FILE STRUCTURE & COMPONENT ROLES

File Path	Module Layer	Technical Responsibility
app.py / appCam.py	BACKEND	Initializes HTTP service, registers application routes, dispatches client control commands.
camera_pi.py	DRIVER	Interacts with the sensor via picamera, buffers captured frames, yields streaming byte generator.
angleServoCtrl.py	HARDWARE	Converts user input angles (0°–180°) into PWM duty cycle values and commands physical GPIO pins.
index.html	FRONTEND	Serves primary monitoring dashboard with an embedded live video player frame.
controller.html	FRONTEND	Provides directional arrow controls, position indicators, and responsive camera adjustment UI.

4. ENGINEERING IMPLEMENTATION & SOFTWARE MODULES

4.1. Video Streaming Pipeline (camera_pi.py & BaseCamera)

To maintain low end-to-end latency on resource-constrained hardware, an MJPEG (Motion JPEG) stream is implemented. A background worker thread inside camera_pi.py queries the camera interface continuously. New frames are compressed into discrete JPEG payloads in memory. The Flask route /video_feed sends these frames using the multipart/x-mixed-replace; boundary=frame MIME type, allowing modern browsers to render continuous video feeds without client-side rendering plugins.

4.2. PWM Motion Control & Mapping (angleServoCtrl.py)

Standard analog servos require a 50 Hz control frequency (20 ms period). The target angle ($0^\circ - 180^\circ$) provided by the user is mapped linearly to the hardware duty cycle using the formula:

$$\text{DutyCycle} = 2.5 + (\text{Target_Angle} / 180.0) * 10.0$$

This mapping yields 2.5% (~0.5 ms pulse) at 0° , 7.5% (~1.5 ms pulse) at 90° center, and 12.5% (~2.5 ms pulse) at 180° , ensuring accurate mechanical alignment without damaging the gear train.

4.3. Responsive Web Control Panel (controller.html)

The template templates/controller.html combines the live video stream with pan-tilt directional triggers. Operators can orient the camera in real time through touch or click events. The client issues asynchronous HTTP requests that are parsed by the backend to reposition the mechanical assembly smoothly.

5. VALIDATION, TESTING & PERFORMANCE BENCHMARKS

Evaluation Parameter	Measured Value	Engineering Assessment
End-to-End Stream Latency	~140 - 220 ms	Provides responsive, real-time visual feedback over local 2.4/5 GHz Wi-Fi.
Stream Resolution & Frame Rate	640x480 @ 24-30 FPS	Balanced profile ensuring sharp fidelity with minimized CPU rendering overhead.
Motor Actuation Latency	< 100 ms	Immediate servo response upon triggering browser-based control events.
Host Resource Utilization	18% - 26% CPU / 65 MB RAM	Lightweight footprint, leaving ample compute margin for auxiliary background tasks.

6. CONCLUSION & ROADMAP

The project successfully achieves reliable, low-overhead video delivery and mechanical directional control through standard web technologies. Planned future extensions include:

- Computer Vision Integration (OpenCV):** Implementing edge-based face or object tracking to support autonomous pan-tilt target following.
- WebRTC Streaming:** Upgrading the streaming protocol from MJPEG to WebRTC to reduce end-to-end transport latency below 50 ms.
- Security & Access Control:** Adding token-based user authentication and HTTPS/TLS encryption to enable secure deployment over public networks.